

Aquí tienes la solución completa al examen modificado:

Examen Final: "La Carrera de Colapinto en la F1" - Soluciones

Ejercicio 1: Listas por Comprensión

```
posiciones = [1, 5, 2, 8, 3, 4, 10, 2, 1]
podios = [pos for pos in posiciones if pos in [1, 2, 3]]
print(podios)  # Salida: [1, 2, 3, 2, 1]
```

Ejercicio 2: Clases y Objetos

```
class AutoF1:
    def __init__(self, equipo, potencia_motor, neumaticos):
        self.equipo = equipo
        self.potencia_motor = potencia_motor
        self.neumaticos = neumaticos

    def tiempo_vuelta(self, adherencia):
        # Suponemos que el tiempo base es inversamente proporcional a la
        # potencia y adherencia
        tiempo_base = 100
        tiempo = tiempo_base / (self.potencia_motor * adherencia)
        return tiempo

# Uso
auto = AutoF1("Equipo A", 500, "blandos")
print(auto.tiempo_vuelta(0.8))  # Ejemplo de cálculo: 0.25 segundos
```

Ejercicio 3: Teoría (Scope y Tiempo de Vida de las Variables)

1. Variables locales y globales:

- Una **variable local** está definida dentro de una función y solo existe mientras la función está en ejecución. Ejemplo:

```
def suma():
    a = 10  # Variable local
    return a + 5
print(suma())  # Salida: 15
```

- Una **variable global** es accesible desde cualquier parte del programa, siempre que no esté redefinida localmente. Ejemplo:

```
b = 20 # Variable global
def resta():
    return b - 5
print(resta()) # Salida: 15
```

2. Si intentas modificar una variable global sin declararla como **global** dentro de una función, Python creará una nueva variable local con el mismo nombre, dejando la variable global sin cambios.

3. **Tiempo de vida:** Es el período durante el cual una variable está disponible en memoria.

- Las variables locales tienen un tiempo de vida limitado al alcance de su función.
- Las variables globales existen durante la ejecución completa del programa.

Ejercicio 4: Recursividad

```
def calcular_puntos(posiciones, n):
    if n == 0:
        return 0
    puntos = {1: 25, 2: 18, 3: 15}
    return puntos.get(posiciones[n-1], 0) + calcular_puntos(posiciones, n-1)

posiciones = [1, 5, 2, 8, 3]
print(calcular_puntos(posiciones, len(posiciones))) # Salida: 58
```

Ejercicio 5: Pilas

```
class Nodo:
    def __init__(self, dato):
        self.dato = dato
        self.siguiente = None

class Pila:
    def __init__(self):
        self.cima = None

    def push(self, dato):
        nuevo = Nodo(dato)
        nuevo.siguiente = self.cima
        self.cima = nuevo

    def pop(self):
```

```

        if self.cima is None:
            return None
        dato = self.cima.dato
        self.cima = self.cima.siguiente
        return dato

# Uso
pila_tareas = Pila()
pila_tareas.push("Cambio de neumáticos")
pila_tareas.push("Ajuste del alerón")
print(pila_tareas.pop()) # Salida: Ajuste del alerón

```

Ejercicio 6: Ordenamiento

```

def ordenar_tiempos(tiempos):
    n = len(tiempos)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if tiempos[j] < tiempos[min_idx]:
                min_idx = j
        tiempos[i], tiempos[min_idx] = tiempos[min_idx], tiempos[i]
    return tiempos

tiempos = [78.5, 75.3, 80.2, 76.4]
print(ordenar_tiempos(tiempos)) # Salida: [75.3, 76.4, 78.5, 80.2]

```

Ejercicio 7: Manejo de Archivos

```

def procesar_resultados(archivo):
    posiciones = {}
    with open(archivo, 'r') as file:
        for linea in file:
            _, posicion = linea.strip().split(',')
            posicion = int(posicion)
            posiciones[posicion] = posiciones.get(posicion, 0) + 1
    return posiciones

# Crea un archivo ejemplo para probar
with open("resultados.txt", "w") as file:
    file.write("carrera1,1\n")
    file.write("carrera2,2\n")
    file.write("carrera3,1\n")

print(procesar_resultados("resultados.txt")) # Salida: {1: 2, 2: 1}

```

Ejercicio 8: Teoría (Complejidad Computacional)

1. El algoritmo de selección busca el menor elemento en una lista y lo coloca en la posición inicial, repitiendo el proceso para las subsecuentes posiciones.
 - Complejidad temporal en el peor caso: $O(n^2)$, ya que hay dos bucles anidados.
 2. QuickSort es más eficiente porque utiliza un enfoque de divide y vencerás, con complejidad promedio $O(n \log n)$, reduciendo el tiempo de ejecución en listas grandes.
-

Ejercicio 9: Teoría (Estructuras de Datos)

1. **Diferencias entre pila y cola:**
 - Una pila sigue la regla LIFO (último en entrar, primero en salir). Ejemplo: Historial de navegación.
 - Una cola sigue la regla FIFO (primero en entrar, primero en salir). Ejemplo: Gestión de procesos en un sistema operativo.
 2. Las listas enlazadas son más eficientes que las listas tradicionales para inserciones y eliminaciones frecuentes porque no requieren mover elementos, mientras que las listas tradicionales deben reasignar memoria y mover datos.
-

Conclusión

Este examen resuelto cubre tanto habilidades prácticas como conocimiento teórico, consolidando conceptos clave de algoritmos y estructuras de datos.